Lesson 2 Teacher's Key

GameMaker

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

In obj_player's Step event:

```
if (keyboard_check(vk_right)) x += 4;
```

Level 2 - Complete Worked Solution

All 4 directions in the Step event:

```
if (keyboard_check(vk_right)) x += 4;
if (keyboard_check(vk_left))  x -= 4;
if (keyboard_check(vk_up))    y -= 4;
if (keyboard_check(vk_down))  y += 4;
```

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking (or if a student blows past all of them, this is your own jumping-off point for exploring further right alongside them).

Animate: blink while moving, static when idle

GameMaker objects have a built-in visible variable that controls whether the Draw event runs automatically - toggling it on and off using a counter incremented in the Step event is the simplest blink, no sprite swap needed. Add blink_timer = 0; to the Create event first:

```
// Step event:
var moving = keyboard_check(vk_right) || keyboard_check(vk_left) ||
keyboard_check(vk_up) || keyboard_check(vk_down);
if (moving) {
    blink_timer += 1;
    visible = (blink_timer mod 20 < 10);
} else {
    visible = true;
    blink_timer = 0;
}
```

Sound: beep-beep while moving, silent when idle

Add sfx_blip.wav as a sound resource (keep its default name, sfx_blip) (right-click > Create Sound). audio_is_playing() accepts the sound asset directly and checks whether any instance of it is currently playing - checking it before calling audio_play_sound() again creates the beep-

beep retrigger rhythm rather than one continuous tone. audio_stop_sound() on the asset is what actually silences it when idle:

```
if (moving) {
    if (!audio_is_playing(sfx_blip)) audio_play_sound(sfx_blip, 1, false);
} else {
    audio_stop_sound(sfx_blip);
}
```

Recolor: different sprite per direction, reset when idle

Same Idle/Moving concept as the State Diagram from this lesson's bonus section. This swaps sprite_index itself rather than tinting with image_blend - import spr_player_red, spr_player_blue, spr_player_green, and spr_player_yellow from this lesson's Assets folder alongside spr_player_orange as your default:

```
if (keyboard_check(vk_right))      sprite_index = spr_player_red;
else if (keyboard_check(vk_left))  sprite_index = spr_player_blue;
else if (keyboard_check(vk_up))    sprite_index = spr_player_green;
else if (keyboard_check(vk_down))  sprite_index = spr_player_yellow;
else                                sprite_index = spr_player_orange; //
idle
```

Clamp movement to the screen

clamp(value, min, max) is a built-in GML function - simplest possible boundary check, no manual if/else needed. Add to the bottom of the Step event:

```
x = clamp(x, 0, room_width);
y = clamp(y, 0, room_height);
```

Structure: group everything triggered by the same action in one place

The Step event already runs once per frame, so the "same place in code" for this lesson is that one event - the trick is organizing what goes inside it, not spreading logic across multiple events or objects. Movement and the room clamp always run, every frame, regardless of direction. Blink, sound, and recolor are all direction-dependent, so they all live under one shared if (moving) / else check - not four separate direction checks scattered through the event. That's the Complete Level 3 solution below.

Complete Level 3 solution (all 5 goals combined)

Everything above combined, across obj_player's two events:

```
// Create event:
blink_timer = 0;

// Step event:
var moving = keyboard_check(vk_right) || keyboard_check(vk_left) ||
keyboard_check(vk_up) || keyboard_check(vk_down);
if (keyboard_check(vk_right)) x += 4;
if (keyboard_check(vk_left))  x -= 4;
if (keyboard_check(vk_up))    y -= 4;
if (keyboard_check(vk_down))  y += 4;

x = clamp(x, 0, room_width);
y = clamp(y, 0, room_height);

if (moving) {
```

```
    blink_timer += 1;
    visible = (blink_timer mod 20 < 10);
    if (!audio_is_playing(sfx_blip)) audio_play_sound(sfx_blip, 1, false);
    if (keyboard_check(vk_right))      sprite_index = spr_player_red;
    else if (keyboard_check(vk_left))  sprite_index = spr_player_blue;
    else if (keyboard_check(vk_up))    sprite_index = spr_player_green;
    else                                sprite_index =
spr_player_yellow; // down
} else {
    visible = true;
    blink_timer = 0;
    audio_stop_sound(sfx_blip);
    sprite_index = spr_player_orange;
}
```

Alternate Path Notes: GML Visual

The Visual toggle is per-event, per-object - a student can mix Code and Visual across different events/objects with no penalty.

Movement uses "Jump To Point" (Movement category) with Relative checked on both X and Y - it's the direct visual equivalent of x += 4 (and y += 4), but it's a purpose-built, self-labeled block rather than the generic Assign Variable, which reads better for a student building entirely in blocks. Leaving Relative unchecked overwrites the position outright, which is a common student mistake to watch for either way.

The Level 3 goals should reach for the same kind of purpose-built block wherever GameMaker has one, instead of defaulting to Assign Variable everywhere: "Set Sprite" (Instance category) for the recolor goal instead of Assign Variable on sprite_index, and "If Audio Is Playing" -> Not -> "Play Audio" (Audio category) as the retrigger guard for the sound goal, with a matching "Stop Audio" in the idle branch (in place of audio_is_playing()/audio_play_sound()/audio_stop_sound()). Assign Variable remains the right - and only - choice for anything that isn't a recognized built-in, like the custom blink_timer or moving variables; the blink itself can still use "Set Instance Variable" -> Visible, since that one is a recognized built-in too.

- Demonstrating both first shows students the two approaches produce identical runtime behavior, which builds intuition for what code "really" does under the hood.
- If a student picks Visual as their primary style for this course, GML Visual remains fully supported all the way through GameMaker's certification track.
- Reaching for the most specific, well-named block available (Jump To Point, Set Sprite, If Audio Is Playing, Play Audio, Stop Audio, Set Instance Variable) before falling back to the generic Assign Variable keeps the block graph readable as plain English - Assign Variable stays available as the fallback for anything without a dedicated block, like a custom variable.